



B3GNL0108

POO JAVA (J2SE, J2ME ET J2EE)

KEVIN NOEL FONKOUA TATANG

CEO LABO ACADEMY, FULL STACK WEB DEVELOPPER
(EH2S Sarl)

B3/S2

PROGRAMME

- 1 Rappels et approfondissement de la POO
- 2 Classes abstraites et interfaces
- 3 Gestion des exceptions
- 4 Introduction aux design patterns en Java

Classes : attributs et méthodes

- Une **classe** est un modèle (blueprint) pour créer des objets.
- Elle contient des **attributs** (variables d'instance) et des **méthodes** (fonctions).

Exemple

```
public class Personne {  
    // attributs  
    String nom;  
    int age;  
  
    // méthode  
    void afficher() {  
        System.out.println("Nom: " + nom + ", Age: " + age);  
    }  
}
```

Encapsulation : private, protected, public

- L'encapsulation protège les données et restreint l'accès.
- **private** : accessible uniquement dans la classe.
- **protected** : accessible dans la classe, les sous-classes et le même package.
- **public** : accessible partout.
- **package-private** (par défaut) : accessible dans le même package.

Exemple

```
public class CompteBancaire {  
    private double solde; // accessible seulement ici  
  
    public void deposer(double montant) {  
        if(montant > 0) solde += montant;  
    }  
}
```

Constructeurs, getters et setters

- Le **constructeur** initialise un nouvel objet.
- Les **getters** et **setters** permettent un accès contrôlé aux attributs privés.

Exemple

```
public class Personne {  
    private String nom;  
    private int age;  
  
    // Constructeur  
    public Personne(String nom, int age) {  
        this.nom = nom;  
        this.age = age;  
    }  
  
    // Getter  
    public String getNom() { return nom; }  
  
    // Setter  
    public void setAge(int age) { this.age = age; }  
}
```

Objets : instantiation et utilisation

- Un **objet** est une instance d'une classe.
- Création avec l'opérateur `new`.

Exemple

```
public class Main {  
    public static void main(String[] args) {  
        Personne p = new Personne("Alice", 25);  
        System.out.println(p.getNom());  
        p.setAge(26);  
    }  
}
```

Héritage

- Permet de créer une nouvelle classe à partir d'une classe existante.
- Utilisation du mot-clé `extends`.
- La classe fille hérite des attributs et méthodes (non privés) de la classe mère.
- `super` fait référence à la classe mère.

Exemple

```
public class Etudiant extends Personne {  
    private String filiere;  
  
    public Etudiant(String nom, int age, String filiere) {  
        super(nom, age); // appel au constructeur parent  
        this.filiere = filiere;  
    }  
}
```

Polymorphisme

- Capacité d'un objet à prendre plusieurs formes.
- **Surcharge** (overload) : plusieurs méthodes de même nom dans une classe, avec des paramètres différents.
- **Redéfinition** (override) : une méthode de la classe fille remplace celle de la classe mère.

Exemple de redéfinition

```
public class Animal {  
    public void faireDuBruit() {  
        System.out.println("Bruit générique");  
    }  
}  
  
public class Chat extends Animal {  
    @Override  
    public void faireDuBruit() {  
        System.out.println("Miaou");  
    }  
}
```


Exercice d'application

Énoncé :

- 1 Créer une classe `Personne` avec les attributs privés `nom` (String) et `age` (int). Ajouter un constructeur, des getters et une méthode `afficher()`.
- 2 Créer une classe `Employe` qui hérite de `Personne` avec un attribut supplémentaire `salaire` (double). Redéfinir `afficher()` pour afficher aussi le salaire.
- 3 Dans un main, créer des objets des deux classes et tester le polymorphisme.

Objectif : manipuler les concepts vus précédemment.

Classe abstraite

- Classe qui ne peut pas être instanciée directement (mot-clé `abstract`).
- Peut contenir des méthodes abstraites (sans corps) et des méthodes concrètes.
- Sert de base pour d'autres classes.

Exemple

```
public abstract class Forme {  
    protected String couleur;  
  
    public abstract double calculerSurface();  
  
    public void afficherCouleur() {  
        System.out.println("Couleur: " + couleur);  
    }  
}
```

Interface

- Contrat que les classes doivent respecter (mot-clé `interface`).
- Toutes les méthodes sont publiques et abstraites (sauf méthodes `default` ou `static` depuis Java 8).
- Une classe peut implémenter plusieurs interfaces (`implements`).

Exemple

```
public interface Dessinable {  
    void dessiner(); // public abstract implicite  
}
```

Particularités : classe abstraite vs interface

Classe abstraite	Interface
Peut avoir des attributs d'instance	Attributs uniquement <code>public static final</code> (constantes)
Peut avoir des méthodes concrètes	Avant Java 8 : uniquement abstraites. Depuis Java 8 : méthodes <code>default</code> et <code>static</code>
Héritage unique (une seule classe parente)	Implémentation multiple possible
Utilisée pour un "est-un" avec du code partagé	Utilisée pour définir un "contrat" ou un "peut être"

Exemple d'application

- Créons une interface `Dessinable` avec une méthode `void dessiner()`.
- Créons une classe abstraite `Forme` implémentant `Dessinable`, avec un attribut `couleur` et une méthode abstraite `double surface()`.
- Créons des classes concrètes `Cercle` et *Rectangle* qui étendent `Forme`.

Extrait

```
public class Cercle extends Forme {  
    private double rayon;  
    public Cercle(String couleur, double rayon) { ... }  
    @Override public double surface() { return Math.PI * rayon * rayon; }  
    @Override public void dessiner() { System.out.println("Dessine un cercle"); }  
}
```

Introduction aux exceptions

- Les exceptions sont des événements qui perturbent le déroulement normal du programme.
- Mécanisme de traitement : `try`, `catch`, `finally`.
- Deux types : **checked** (vérifiées à la compilation) et **unchecked** (`RuntimeException`).

Structure

```
try {  
    // code susceptible de lever une exception  
} catch (TypeException e) {  
    // traitement  
} finally {  
    // toujours exécuté (optionnel)  
}
```

Exemples concrets

Division par zéro (unchecked)

```
int a = 10, b = 0;
try {
    int resultat = a / b;
} catch (ArithmeticException e) {
    System.out.println("Division par zéro !");
}
```

Fichier non trouvé (checked)

```
try {
    FileReader file = new FileReader("fichier.txt");
} catch (FileNotFoundException e) {
    System.out.println("Fichier introuvable.");
}
```

Bonnes pratiques

- Ne pas ignorer les exceptions (catch vide).
- Utiliser plusieurs blocs `catch` pour traiter différentes exceptions.
- Préférer les exceptions spécifiques à `Exception` générale.
- Libérer les ressources dans `finally` ou utiliser try-with-resources (Java 7+).
- Propager l'exception si on ne peut pas la traiter (avec `throws`).

Qu'est-ce qu'un design pattern ?

- Solution éprouvée à un problème récurrent dans la conception de logiciels.
- Ce ne sont pas des morceaux de code, mais des descriptions de solutions.
- Trois grandes catégories :
 - **Création** : concernent l'instanciation des objets (Singleton, Factory, Builder...)
 - **Structure** : organisent les classes et objets (Adapter, Composite, Proxy...)
 - **Comportement** : gèrent les interactions et responsabilités (Observateur, Stratégie, Template...)

Exemple : Singleton

- Garantit qu'une classe n'a qu'une seule instance et fournit un point d'accès global.

Implémentation simple (thread-safe)

```
public class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {} // constructeur privé  
  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Exemple : Factory (méthode de fabrication)

- Définit une interface pour créer un objet, mais laisse les sous-classes décider quelle classe instancier.

Exemple simplifié

```
public interface Animal { void parler(); }  
public class Chien implements Animal { public void parler() { System.out.p  
rintln("Ouaaf"); } }  
public class Chat implements Animal { public void parler() { System.out.pr  
intln("Miaou"); } }  
  
public class AnimalFactory {  
    public static Animal getAnimal(String type) {  
        if ("chien".equals(type)) return new Chien();  
        else if ("chat".equals(type)) return new Chat();  
        else return null;  
    }  
}
```

Exemple : Observateur

- Permet à des objets d'être notifiés des changements d'état d'un autre objet.

Extrait (java.util.Observer, obsolète mais illustratif)

```
import java.util.Observable;
import java.util.Observer;

public class Meteo extends Observable {
    private float temperature;
    public void setTemperature(float temp) {
        this.temperature = temp;
        setChanged();
        notifyObservers(temp);
    }
}

public class Affichage implements Observer {
    public void update(Observable o, Object arg) {
```

Conclusion

- La POO est la base de Java.
- Les classes abstraites et interfaces sont essentielles pour une conception flexible.
- La gestion des exceptions rend les programmes robustes.
- Les design patterns sont des outils précieux pour résoudre des problèmes courants.

Bonne continuation !

TP PAR Mr KEVIN FONKOUA